

# **CAMPUS PRODUCTIVITY TOOL**

## **PHASE I REPORT**

*Submitted by*

|                     |                  |
|---------------------|------------------|
| <b>KIRUTHIKA KM</b> | <b>230701153</b> |
| <b>LAKSHAYA S</b>   | <b>230701160</b> |
| <b>MOUNIKKHA GA</b> | <b>230701198</b> |
| <b>MADHUNESHA G</b> | <b>230701170</b> |

*in partial fulfillment for the award of the degree*

**of**

**BACHELOR OF ENGINEERING**

**IN**

**COMPUTER SCIENCE AND ENGINEERING**



**RAJALAKSHMI ENGINEERING COLLEGE, CHENNAI**

**ANNA UNIVERSITY: CHENNAI 600 025**

**OCTOBER 2026**

**ANNA UNIVERSITY, CHENNAI**  
**BONAFIDE CERTIFICATE**

Certified that this Report titled “**CAMPUS PRODUCTIVITY TOOL**” is the bonafide work of **KIRUTHIKA KM (230701153), LAKSHAYA S (230701160), MOUNIKKHA GA (230701198), MADHUNESHA G(230701170)** who carried out the work under my supervision. Certified further that to the best of my knowledge the work reported herein does not form part of any other thesis or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidates.

“This project addresses the following Sustainable Development Goal(s): Goal No. 4: Quality Education.”

**SIGNATURE**

Dr. J. Manoranjini,  
Professor and Head,  
Department of Computer Science  
and Engineering,  
Rajalakshmi Engineering College,  
Chennai – 602 105.

**SIGNATURE**

Mr.J.M Akshay Khanth,  
Assistant Professor,  
Department of Computer Science  
and Engineering,  
Rajalakshmi Engineering College,  
Chennai – 602 105.

Submitted to Project Viva-Voce Examination held on \_\_\_\_\_

**Internal Examiner**

**External Examiner**

## **ABSTRACT**

The Student Portal Management System is a role-based web application developed to manage and integrate academic, student, teacher, examination, and placement activities through a centralized platform. The system provides separate functionalities for Admin, Teacher, and Student users. Admins can manage student and teacher details, campus interview schedules, and company profiles with eligibility criteria based on 10th and 12th-grade percentages. Teachers can manage student timetables and attendance records, question papers and answers. Students can view their academic information, upload documents for RAG-based text summarization and MCQ generation, and upload resumes for company eligibility analysis. The system also provides email alerts for upcoming campus interviews and supports automated answer-sheet evaluation by comparing student answers with stored answers using similarity-based analysis. The answer analysis is expected to achieve approximately 80–90% effectiveness, depending on document quality and answer similarity. Overall, the system aims to simplify academic management, support AI-assisted learning, automate basic assessment processes, and improve campus placement coordination through a single intelligent web-based platform.

## **ACKNOWLEDGEMENT**

Initially we thank the Almighty for being with us through every walk of our life and showering his blessings through the endeavor to put forth this report. Our sincere thanks to our Chairman Mr. S. MEGANATHAN, B.E, F.I.E., our Vice Chairman Mr. ABHAY SHANKAR MEGANATHAN, B.E., M.S., and our respected Chairperson Dr. (Mrs.) THANGAM MEGANATHAN, Ph.D., for providing us with the requisite infrastructure and sincere endeavoring in educating us in their premier institution.

Our sincere thanks to Dr. S.N. MURUGESAN, M.E., Ph.D., our beloved Principal for his kind support and facilities provided to complete our work in time. We express our sincere thanks to Dr. J. Manoranjini, Ph.D., Professor and Head of the Department of Computer Science and Engineering for the guidance and encouragement throughout the project work. We convey our sincere and deepest gratitude to our internal guide, Mr.AKSHAY KHANTH JM, Assistant Professor , Department of Computer Science and Engineering for the valuable guidance throughout the course of the project. We thank our Project Coordinator, Dr.S.MADHUSUDHANAN, AOP, Department of Computer Science and Engineering for the useful tips during our review to build our project.

|                     |                    |
|---------------------|--------------------|
| <b>KIRUTHIKA KM</b> | <b>: 230701153</b> |
| <b>LAKSHAYA S</b>   | <b>: 230701160</b> |
| <b>MOUNIKKHA GA</b> | <b>: 230701198</b> |
| <b>MADHUNESHA G</b> | <b>: 230701170</b> |

## TABLE OF CONTENTS

| CHAPTER NO. | TITLE                                          | PAGE NO.    |
|-------------|------------------------------------------------|-------------|
|             | <b>ABSTRACT .....</b>                          | <b>iii</b>  |
|             | <b>LIST OF FIGURES.....</b>                    | <b>vii</b>  |
|             | <b>LIST OF ABBREVIATIONS.....</b>              | <b>viii</b> |
| <b>1</b>    | <b>INTRODUCTION.....</b>                       | <b>1</b>    |
| 1.1         | GENERAL .....                                  | 1           |
| 1.2         | SCOPE OF THE PROJECT .....                     | 1           |
| 1.3         | OBJECTIVE.....                                 | 2           |
| 1.4         | MOTIVATION OF THE PROJECT .....                | 2           |
| 1.5         | PROBLEM STATEMENT .....                        | 2           |
| 1.6         | EXISTING SYSTEM .....                          | 3           |
| 1.7         | PROPOSED SYSTEM .....                          | 4           |
| <b>2</b>    | <b>LITERATURE SURVEY .....</b>                 | <b>6</b>    |
| <b>3</b>    | <b>SYSTEM DESIGN.....</b>                      | <b>9</b>    |
| 3.1         | ARCHITECTURE DIAGRAM.....                      | 9           |
| 3.2         | MODULE DESCRIPTION .....                       | 9           |
| 3.2.1       | ADMIN MODULE .....                             | 10          |
| 3.2.2       | TEACHER MODULE .....                           | 10          |
| 3.2.3       | STUDENT MODULE.....                            | 10          |
| 3.2.4       | AI-BASED DOCUMENT PROCESSING MODULE .....      | 10          |
| 3.2.5       | EXAMINATION AND ANSWER EVALUATION MODULE ..... | 11          |
| 3.2.6       | CAMPUS PLACEMENT MANAGEMENT MODULE .....       | 11          |
| 3.3         | UML DIAGRAMS .....                             | 11          |
| 3.3.1       | USE CASE AND ACTIVITY DIAGRAMS.....            | 11          |
| 3.3.2       | DATA FLOW DIAGRAM.....                         | 13          |
| 3.3.3       | OVERALL DIAGRAM .....                          | 14          |
| 3.4         | SYSTEM REQUIREMENTS.....                       | 14          |
| 3.4.1       | HARDWARE REQUIREMENTS.....                     | 14          |
| 3.4.2       | SOFTWARE REQUIREMENTS .....                    | 15          |
| 3.5         | FEASIBILITY STUDY .....                        | 15          |
| 3.5.1       | ECONOMICAL FEASIBILITY.....                    | 15          |
| 3.5.2       | TECHNICAL FEASIBILITY.....                     | 15          |
| 3.5.3       | SOCIAL FEASIBILITY .....                       | 15          |
| <b>4</b>    | <b>PROPOSED SYSTEM ALGORITHM.....</b>          | <b>16</b>   |
| 4.1         | LARGE LANGUAGE MODEL (LLM).....                | 16          |

|          |                                                  |           |
|----------|--------------------------------------------------|-----------|
| 4.1.1    | WORKING OF LLM .....                             | 16        |
| 4.1.2    | POPULAR LLMs .....                               | 16        |
| 4.2      | RETRIEVAL-AUGMENTED GENERATION (RAG) .....       | 17        |
| 4.2.1    | WORKING OF RAG .....                             | 17        |
| 4.2.2    | POPULAR RAG TECHNOLOGIES AND MODELS .....        | 18        |
| 4.3      | PYTHON PROGRAMMING LANGUAGE .....                | 18        |
| 4.3.1    | INTRODUCTION TO PYTHON .....                     | 18        |
| 4.3.2    | WHY PYTHON? .....                                | 18        |
| 4.3.3    | REAL-WORLD EXAMPLES .....                        | 19        |
| 4.3.4    | APPLICATIONS OF PYTHON .....                     | 19        |
| 4.3.5    | PYTHON STANDARD LIBRARY AND PACKAGES .....       | 19        |
| <b>5</b> | <b>IMPLEMENTATION AND RESULT DISCUSSION.....</b> | <b>21</b> |
| 5.1      | IMPLEMENTATION.....                              | 21        |
| 5.1.1    | IMPLEMENTATION OF ROLE-BASED MODULES .....       | 21        |
| 5.1.2    | IMPLEMENTATION OF DOCUMENT PROCESSING .....      | 21        |
| 5.1.3    | IMPLEMENTATION OF ANSWER EVALUATION .....        | 21        |
| 5.1.4    | IMPLEMENTATION OF PLACEMENT SERVICES .....       | 21        |
| 5.2      | SOFTWARE TESTING.....                            | 22        |
| 5.2.1    | UNIT TESTING .....                               | 22        |
| 5.2.2    | INTEGRATION TESTING.....                         | 22        |
| 5.2.3    | FUNCTIONAL TESTING .....                         | 22        |
| 5.2.4    | SYSTEM TESTING .....                             | 23        |
| 5.2.5    | WHITE BOX TESTING .....                          | 23        |
| 5.2.6    | BLACK BOX TESTING .....                          | 23        |
| 5.3      | DISCUSSION.....                                  | 23        |
| <b>6</b> | <b>CONCLUSION AND FUTURE ENHANCEMENT.....</b>    | <b>24</b> |
| 6.1      | CONCLUSION .....                                 | 24        |
| 6.2      | FUTURE ENHANCEMENT .....                         | 24        |
|          | <b>REFERENCES .....</b>                          | <b>25</b> |

## LIST OF FIGURES

| FIGURE NO. | TITLE                                             | PAGE NO. |
|------------|---------------------------------------------------|----------|
| 1.1        | Overview of the Proposed System Architecture..... | 4        |
| 3.1        | Architecture Diagram for the Entire System .....  | 9        |
| 3.2        | Use Case Diagram.....                             | 12       |
| 3.3        | Activity Diagram.....                             | 13       |
| 3.4        | Data Flow Diagram – Level 0.....                  | 13       |
| 3.5        | Data Flow Diagram – Level 1.....                  | 14       |
| 3.6        | Data Flow Diagram – Level 2.....                  | 14       |
| 3.7        | Overall Diagram of the System.....                | 14       |

## LIST OF ABBREVIATIONS

| Abbreviation | Expansion                          |
|--------------|------------------------------------|
| AI           | Artificial Intelligence            |
| ATS          | Automatic Text Summarization       |
| DFD          | Data Flow Diagram                  |
| GUI          | Graphical User Interface           |
| IDE          | Integrated Development Environment |
| LLM          | Large Language Model               |
| MCQ          | Multiple Choice Question           |
| ML           | Machine Learning                   |
| NLG          | Natural Language Generation        |
| NLU          | Natural Language Understanding     |
| PyPI         | Python Package Index               |
| RAG          | Retrieval-Augmented Generation     |
| UML          | Unified Modeling Language          |

# **CHAPTER 1**

## **INTRODUCTION**

### **1.1 GENERAL**

The Student Portal Management System is a centralized role-based web application designed to integrate and simplify various academic and placement-related activities within an educational institution. The system provides separate functionalities for Admin, Teacher, and Student users, allowing each role to perform their respective activities through a unified platform. Admins can manage student and teacher information, company details, eligibility criteria, and campus interview schedules, while teachers can maintain student timetables, attendance records, question papers, and answers. Students can access their academic information, upload study documents for RAG-based text summarization and MCQ generation, and upload resumes for automated company eligibility analysis. The system also supports email notifications for upcoming campus interviews and similarity-based automated answer-sheet evaluation. By combining conventional academic management features with AI-assisted functionalities, the proposed system provides an efficient and intelligent platform for managing student activities, supporting personalized learning, and improving placement coordination.

### **1.2 SCOPE OF THE PROJECT**

The scope of the Student Portal Management System covers the centralized management of academic, examination, learning, and placement activities in an educational institution. The system enables Admins to manage student and teacher profiles, company information, interview schedules, and eligibility requirements based on academic performance. Teachers can maintain student timetables and attendance, prepare question papers, and store correct answers for assessment purposes. Students can view their academic details and use AI-assisted learning features by uploading study materials for RAG-based text summarization and MCQ generation. The system also provides resume-based company eligibility analysis to help students identify suitable placement opportunities. Automated email alerts can notify students about

upcoming campus interviews, while answer-sheet evaluation can compare student responses with stored answers using similarity-based analysis. Thus, the system can serve as a single platform for improving academic administration, student learning support, examination management, and placement coordination.

### **1.3 OBJECTIVE**

The main objective of the Student Portal Management System is to develop an integrated web-based platform that simplifies academic management and enhances student learning and placement activities. The system aims to provide role-based access for Admin, Teacher, and Student users while allowing efficient management of academic and institutional information. Another objective is to support students with AI-based document summarization and MCQ generation using RAG techniques, enabling them to learn from their own study materials. The system also aims to automate basic examination activities through similarity-based answer-sheet evaluation and assist students in identifying companies for which they satisfy the required eligibility criteria. Additionally, the system provides timely email alerts for campus interviews, thereby reducing manual communication and improving placement coordination. Overall, the objective is to reduce repetitive administrative work, improve accessibility of academic information, and provide intelligent features that support students throughout their academic and placement journey.

### **1.4 MOTIVATION OF THE PROJECT**

The motivation for developing the Student Portal Management System comes from the need to reduce the dependency on multiple disconnected processes for managing student academic, examination, learning, and placement activities. In many educational environments, student information, attendance, timetables, examination materials, placement details, and interview notifications may be maintained through separate systems or manual procedures, making information management time-consuming and less efficient. Students may also face difficulties in understanding large study materials and identifying important topics for examination preparation. To address these challenges, the proposed system incorporates RAG-based text summarization and MCQ generation, allowing students to obtain useful learning content directly from their uploaded documents. Automated answer evaluation and resume-based eligibility analysis further reduce repetitive manual work and provide

faster assistance to students. These requirements motivate the development of a centralized and intelligent portal that can improve academic support, simplify institutional management, and make placement-related activities more organized.

## **1.5 PROBLEM STATEMENT**

Educational institutions often manage student academic information, attendance, timetables, examinations, learning materials, and placement activities through separate processes, which can result in increased manual effort, information duplication, and difficulty in accessing required information. Teachers may need to maintain student records and evaluate answer sheets manually, while students may have limited support for quickly understanding lengthy study materials and preparing practice questions from their own documents. Placement activities can also require students to manually check company eligibility criteria and keep track of upcoming campus interviews. These limitations can lead to inefficient management, delayed communication, and additional workload for teachers and administrators. Therefore, there is a need for a centralized Student Portal Management System that integrates academic and placement management with AI-assisted learning and automation features. The proposed system addresses these problems by providing role-based management, document-based summarization and MCQ generation, resume eligibility analysis, automated interview notifications, and similarity-based answer evaluation within a single web-based platform.

## **1.6 EXISTING SYSTEM**

The existing system for managing student-related activities is generally based on separate manual or basic digital processes for academic, examination, attendance, and placement activities. Student and teacher details may be maintained independently, while attendance, timetables, question papers, and answer sheets are often handled manually or through different applications. Placement information such as company profiles, eligibility criteria, and interview schedules may also be managed separately, requiring students and staff to track information manually. In such systems, there is limited integration of AI-based learning features, document summarization, automatic MCQ generation, resume-based eligibility analysis, and automated answer evaluation. As a result, students and teachers may need to spend more time managing information and performing repetitive academic and placement-related tasks.

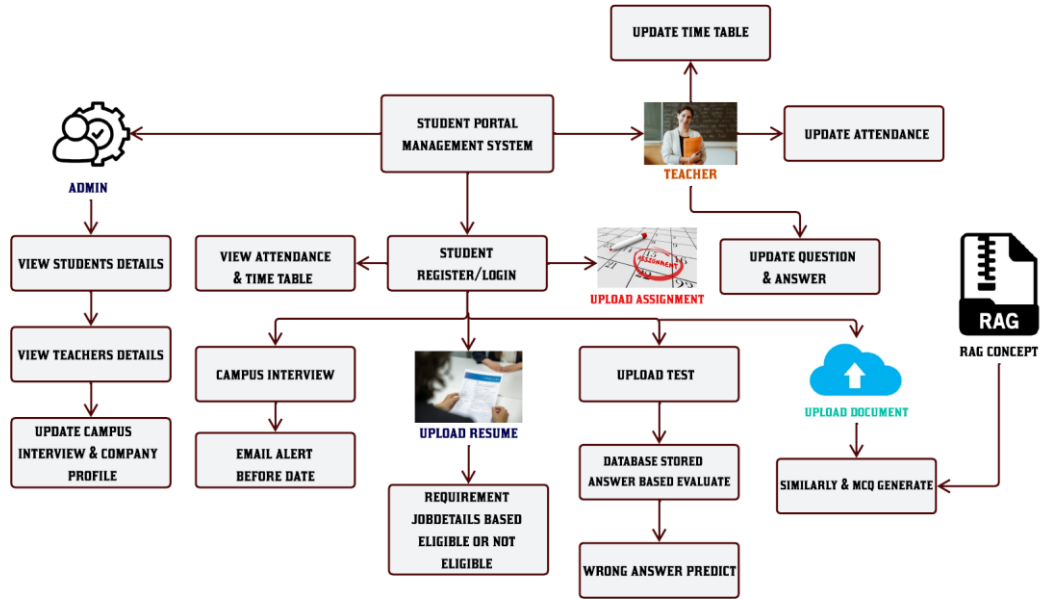
## **Disadvantages of the Existing System**

- Academic, examination, attendance, and placement activities are often managed through separate systems or manual processes.
- Maintaining student records, attendance, timetables, and examination details manually is time-consuming and may lead to errors.
- Students have limited support for quickly summarizing study materials and generating practice questions.
- Placement eligibility checking and campus interview notifications may require manual tracking, which can cause delays or missed opportunities.

## **1.7 PROPOSED SYSTEM**

The proposed Student Portal Management System is a role-based web application designed to manage academic, examination, and placement activities through a centralized platform. The system provides separate access for Admin, Teacher, and Student users. The Admin can manage student and teacher details, store corresponding answers, update campus interview schedules, and maintain company profiles with eligibility criteria based on 10th and 12th-grade percentages. Teachers can create and manage student timetables, update attendance records, and prepare question papers through the system. Students can view their timetable and attendance details and access various AI-based learning and placement services.

The proposed system integrates Retrieval-Augmented Generation (RAG) for document-based processing. Students can upload documents and generate meaningful text summaries and multiple-choice questions based on the uploaded content. The system also supports resume-based eligibility analysis by comparing students' academic percentages with company eligibility requirements stored by the Admin. Email notifications are provided to students before scheduled campus interview dates. In addition, students can upload test answer-sheet documents, which are processed and compared with the correct answers stored in the database using similarity-based analysis. The system identifies correctly and incorrectly answered questions with an expected effectiveness of approximately 80–90%, depending on document quality and answer similarity. Overall, the proposed system reduces manual work, improves academic monitoring, supports AI-assisted learning, and streamlines campus placement management. The overview of the proposed system is presented in Figure 1.1.



**Figure 1.1. Overview of the Proposed System Architecture**

As seen in Figure 1.1, the Student Portal Management System is organised around three role-based users. The Admin views student and teacher details and updates the campus interview and company profile. The Teacher updates the timetable, attendance, and the questions and answers. The Student registers or logs in to view attendance and timetable, uploads assignments, receives email alerts before campus interviews, uploads a resume for eligibility checking, uploads a test for database-based answer evaluation and wrong-answer prediction, and uploads documents for RAG-based similarity analysis and MCQ generation.

### Advantages of the Proposed System

- Provides a centralized platform for managing student, teacher, examination, attendance, and placement activities, reducing manual work.
- Supports AI-based document processing to generate summaries and MCQs, helping students with personalized learning.
- Automates placement activities through resume-based eligibility checking and email notifications for campus interviews.
- Enables automated answer-sheet evaluation using similarity-based analysis, reducing manual correction time and improving evaluation efficiency.

## **CHAPTER 2**

### **LITERATURE SURVEY**

This chapter reviews existing research related to the major functions of the proposed system, namely automatic text summarization, automated multiple-choice question generation, chatbot-based student advising, and data preparation for conversational systems. The study of these works helps in understanding the techniques currently used and the way they support AI-assisted learning.

#### **2.1 Exploring the Landscape of Automatic Text Summarization: A Comprehensive Survey**

*Authors: Bilal Khan, Zohaib Ali Shah, Muhammad Usman, Inayat Khan, Badam Niazi (2023)*

Automatic Text Summarization (ATS) is a rapidly growing field that aims to save readers time and effort by automatically generating summaries of large volumes of text. In recent years, significant advancements have been witnessed in this area, accompanied by challenges that have spurred extensive research. The proliferation of textual data has sparked substantial interest in ATS, which is thoroughly examined in this survey study. Researchers have been refining ATS techniques since the 1950s, primarily categorized as extractive, abstractive, or hybrid approaches. In the extractive approach, key sentences are extracted from the source document(s) and combined to form the summary, while the abstractive approach employs an intermediary representation of the input document(s) to generate a summary that may differ from the original text. Hybrid approaches combine elements of both extractive and abstractive methods. Despite various recommended methodologies, the generated summaries still exhibit noticeable differences compared to those created by humans. This research survey offers an inclusive exploration of ATS, covering its challenges, types, classifications, approaches, applications, methods, implementations, processing and preprocessing techniques, linguistic analysis, datasets, and evaluation measures, catering to the needs of researchers in the field.

#### **2.2 Two-Stage Text Summary Model**

*Authors: Yang Chen, Roben A. Juanatas (2024)*

In order to solve the problems of redundant information processing and high quality summary generation in existing methods, this paper proposes a two-stage text

summary model which is composed of abstracted and generated models. First of all, the important information is abstracted by using an abstracted model which incorporates dilated convolution and gated convolution. Then, a replication mechanism is incorporated into the generated model to ensure that both primary and secondary information are taken into consideration, while also optimizing the cluster search algorithm. Finally, the network structure is reconfigured in the generated model to effectively integrate the coding capabilities of the two-way language model and the text generation abilities of the one-way language model. Experiments were conducted on the CNewSum dataset, and Rouge-1, Rouge-2, and Rouge-L values of 44.21, 27.52, and 39.03 were achieved for the two-paragraph text summary model respectively. The results indicate a significant enhancement in the performance of the two-paragraph text summary model compared to the benchmark model.

### **2.3 MCQGen: A Large Language Model-Driven MCQ Generator for Personalized Learning**

*Authors: Ching Nam Hang, Chee Wei Tan, Pei-Duo Yu (2024)*

In the dynamic landscape of contemporary education, the evolution of teaching strategies such as blended learning and flipped classrooms has highlighted the need for efficient and effective generation of multiple-choice questions (MCQs). To address this, MCQGen, a novel generative artificial intelligence framework designed for the automated creation of MCQs, is introduced. MCQGen uniquely integrates a large language model (LLM) with retrieval-augmented generation and advanced prompt engineering techniques, drawing from an extensive external knowledge base. This integration significantly enhances the ability of the LLM to produce educationally relevant questions that align with both the goals of educators and the diverse learning needs of students. The framework employs innovative prompt engineering, combining chain-of-thought and self-refine prompting techniques, to enhance the performance of the LLM. This process leads to the generation of questions that are not only contextually relevant and challenging but also reflective of common student misconceptions, contributing effectively to personalized learning experiences and enhancing student engagement and understanding. The evaluations showcase the effectiveness of MCQGen in producing high-quality MCQs for various educational needs and learning styles. The framework demonstrates its potential to significantly reduce the time and expertise required for MCQ creation, marking its practical utility in modern education.

## **2.4 Assessing the Impact of Chatbot-Human Personality Congruence on User Behavior: A Chatbot-Based Advising System Case**

*Authors: Mohammad Amin Kuhail, Mohamed Bahja, Ons Al-Shamaileh, Justin Thomas, Amina Alkazemi, Joao Negreiros (2024)*

Chatbot personality has been demonstrated to influence user behavior, such as trust and intended engagement. However, previous research on chatbot-user personality congruence's influence on user behavior is scant despite its significance in human-human conversations. This study explores the effect of chatbot-human personality trait congruence on user behavior in the context of a chatbot-based advising system. In the study, 54 college students interacted with chatbots with three different personalities (extraversion, agreeableness, and conscientiousness) and rated their trust, usage intention, and intended engagement with the chatbots. Additionally, 18 participants were interviewed to gain further insights into their perceptions and evaluations of the chatbots. The findings show that chatbot-human personality congruence influenced behavior, particularly among extroverted participants. No significant differences were observed in the influence of chatbot-human personality congruence on the behavior of conscientious and agreeable participants. Implications and future research directions are presented.

## **2.5 Data Augmentation and Preparation Process of PerInfEx: A Persian Chatbot With the Ability of Information Extraction**

*Authors: Pegah Safari and Mehrnoush Shamsfard (2024)*

This paper describes the data preparation for the proposed chatbot PerInfEx (Persian Information Extraction chatbot). It aims to interactively chit-chat with users in Persian and, by asking the least number of direct questions, extract as much personal information as possible such as the user's age or occupation. Collecting data in considerable size and aligned with the system's specifics is a crucial step to train data-hungry modules of Natural Language Understanding (NLU) and Natural Language Generation (NLG). Initially, for the NLU module, 99 free-discussion dialogues were collected and 74 English training conversations were crawled as more general datasets, while 72 dialogues of the ConvAI2 corpus were manually translated. Moreover, the collection was gamified by implementing a chatting website which resulted in 94 dialogues. Two augmentation methods are proposed: a semi-automatic and a novel fully automatic method, comprehensively evaluated on NLU benchmarks and applied on the

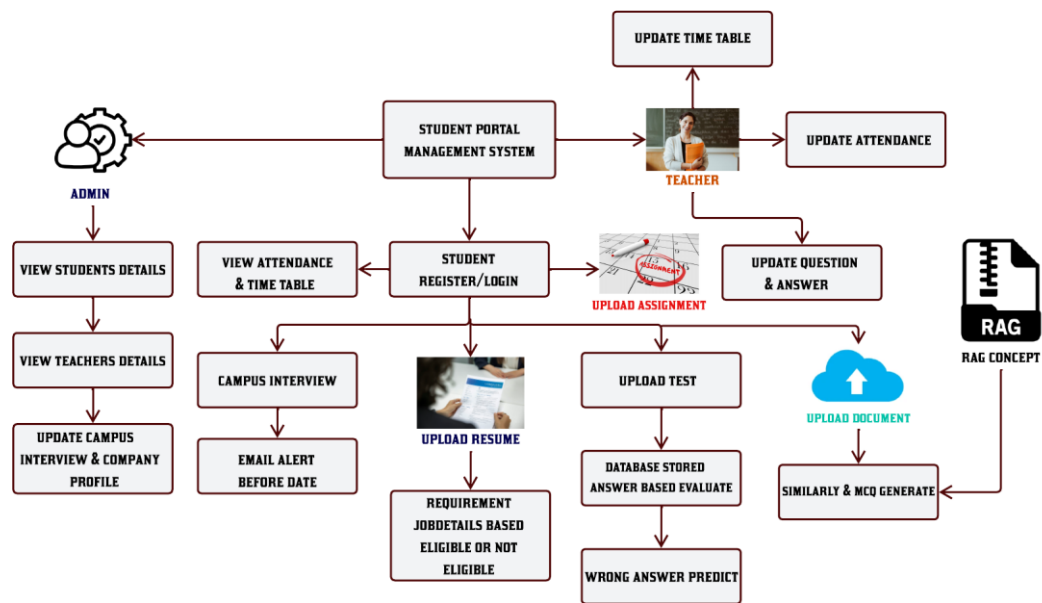
datasets. By prompting OpenAI's GPT-3.5 model, 304 dialogues were also generated automatically. The first part of these datasets is manually annotated while an active learning method is used for annotating the rest. To evaluate data quality, the data are assessed extrinsically using an NLU baseline, which results in intent-accuracy = 88.64, slot-F1 = 83.68 and exact-match = 78.22. For the NLG module, almost the rest of the ConvAI2 corpus (16,217 dialogues) was automatically translated and the previous sets were paraphrased for fine-tuning using the GPT-3.5 model. Their assessment using the NLG baseline results in a perplexity of 15.74 on the train set and 52.17 on the test set.

## CHAPTER 3

### SYSTEM DESIGN

#### 3.1 ARCHITECTURE DIAGRAM

The Student Portal Management System is designed as a role-based web application in which the Admin, Teacher, and Student users access their respective services through a central portal. The architecture of the entire system is presented in Figure 3.1.



**Figure 3.1. Architecture Diagram for the Entire System**

The Admin module is used to view student and teacher details and to update the campus interview and company profile. The Teacher module is used to update the timetable and attendance and to update the questions and answers. The Student module begins with registration or login and offers services to view the attendance and timetable, to upload assignments, to receive an email alert before the campus interview date, to upload a resume for an eligible or not eligible decision based on the job requirements, to upload a test whose answers are evaluated against the database-stored answers so that wrong answers can be predicted, and to upload a document for similarity analysis and MCQ generation based on the RAG concept.

## **3.2 MODULE DESCRIPTION**

The system is divided into the following modules, each of which handles a specific set of responsibilities:

- Admin Module
- Teacher Module
- Student Module
- AI-Based Document Processing Module
- Examination and Answer Evaluation Module
- Campus Placement Management Module

### **3.2.1 ADMIN MODULE**

The Admin Module manages the overall system and maintains student and teacher details. Admin can manage campus interview schedules and company profiles. Company eligibility criteria based on 10th and 12th-grade percentages can be added and updated. The module also stores correct answers for examination and answer evaluation. Admin can monitor and coordinate the major activities of the portal.

### **3.2.2 TEACHER MODULE**

The Teacher Module allows teachers to manage academic-related activities. Teachers can create and update student timetables and maintain attendance records. They can prepare question papers and store the corresponding answers in the system. Teachers can manage examination-related information through the portal. This module helps reduce manual academic record management.

### **3.2.3 STUDENT MODULE**

The Student Module provides students with access to their academic and placement-related information. Students can view their timetable and attendance details. They can upload study documents for AI-based processing and generate summaries and MCQs. Students can also upload resumes for company eligibility analysis. The module provides a simple interface for students to access all available portal services.

### **3.2.4 AI-BASED DOCUMENT PROCESSING MODULE**

The AI-Based Document Processing Module processes documents uploaded by students using RAG-based techniques. The system extracts meaningful information from the uploaded content and generates concise text summaries. It can also generate multiple-choice questions based on the document content. This module helps students understand and practice their study materials. The generated content is based on the information available in the uploaded documents.

### **3.2.5 EXAMINATION AND ANSWER EVALUATION MODULE**

The Examination and Answer Evaluation Module manages question papers, correct answers, and student answer sheets. Students can upload their completed answer-sheet documents through the system. The uploaded answers are processed and compared with the correct answers stored in the database using similarity-based analysis. The system identifies correctly and incorrectly answered questions and provides an automated evaluation result. The expected effectiveness is approximately 80–90%, depending on document quality and answer similarity.

### **3.2.6 CAMPUS PLACEMENT MANAGEMENT MODULE**

The Campus Placement Management Module helps manage company profiles, eligibility criteria, and campus interview schedules. Students' academic percentages can be compared with the eligibility requirements of available companies. The system helps identify companies for which a student is eligible based on their academic details. Email alerts are provided to students before scheduled campus interviews. This module improves placement coordination and reduces manual communication.

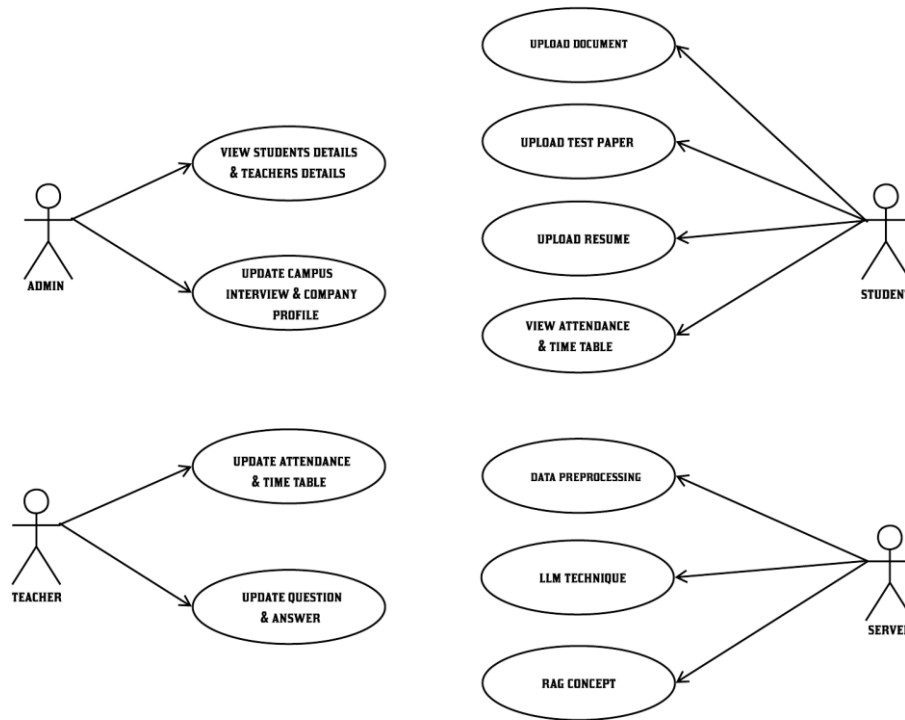
## **3.3 UML DIAGRAMS**

Unified Modeling Language (UML) diagrams are used to represent the structure and behaviour of the system. The use case diagram, activity diagram, data flow diagrams, and the overall diagram of the Student Portal Management System are presented in this section.

### **3.3.1 USE CASE AND ACTIVITY DIAGRAMS**

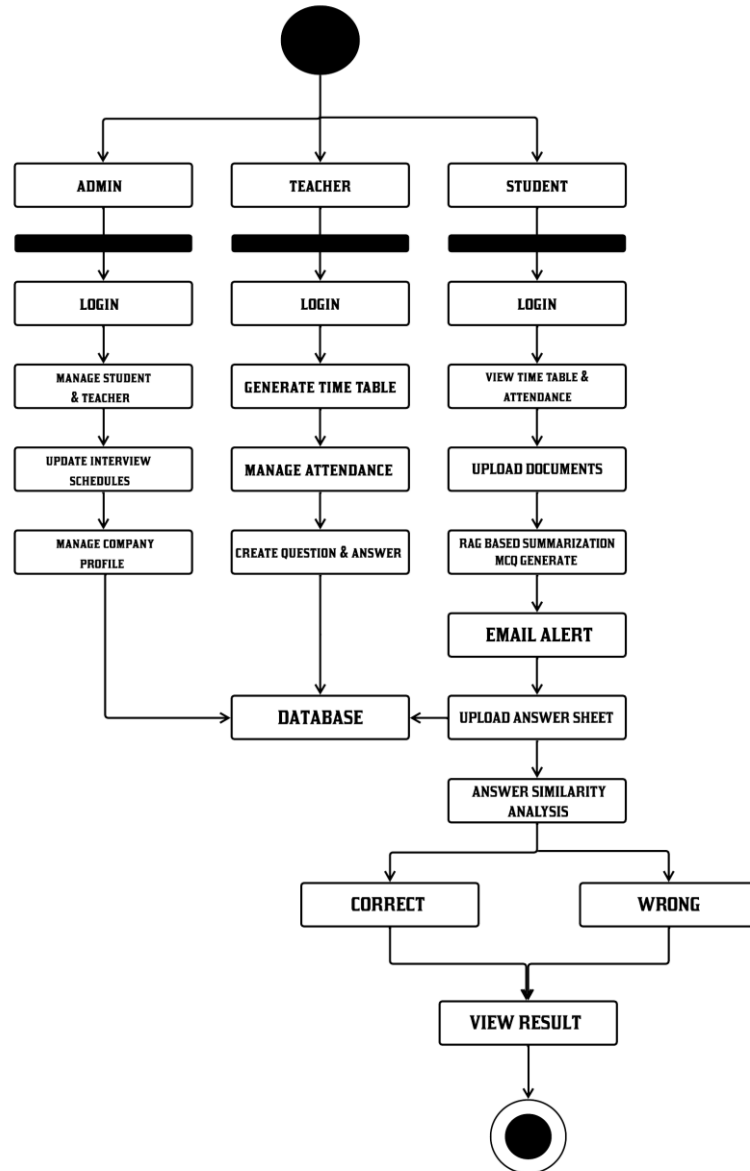
The use case diagram in Figure 3.2 shows the interaction between the actors and the system. The Admin can view student and teacher details and update the campus

interview and company profile. The Teacher can update the attendance and timetable and update the questions and answers. The Student can upload documents, upload test papers, upload resumes, and view the attendance and timetable. The Server performs data preprocessing and applies the LLM technique and the RAG concept.



**Figure 3.2. Use Case Diagram**

The activity diagram in Figure 3.3 presents the flow of activities for each user of the system. The Admin, Teacher, and Student log in and perform their respective activities. The Admin manages student and teacher details, updates interview schedules, and manages the company profile. The Teacher generates the timetable, manages attendance, and creates questions and answers. The Student views the timetable and attendance, uploads documents for RAG-based summarization and MCQ generation, receives an email alert, and uploads the answer sheet. The data from all the users are stored in the database. The answer sheet undergoes answer similarity analysis, the answers are classified as correct or wrong, and the result is displayed to the student.



**Figure 3.3. Activity Diagram**

### 3.3.2 DATA FLOW DIAGRAM

A data flow diagram depicts the flow of data between the different components of the system. Data flow diagrams can be drawn in several levels. The Level 0, Level 1, and Level 2 data flow diagrams of the system are given in Figures 3.4, 3.5, and 3.6 respectively.

## Level 0



**Figure 3.4. Data Flow Diagram – Level 0**

In Level 0, the managing of student and teacher details leads to the updating of interview schedules, which in turn leads to the management of the company profile.

## Level 1



**Figure 3.5. Data Flow Diagram – Level 1**

In Level 1, the generated timetable is followed by attendance management, which leads to the creation of questions and answers.

## Level 2

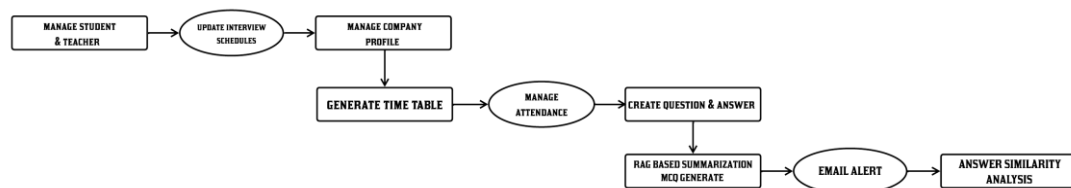


**Figure 3.6. Data Flow Diagram – Level 2**

In Level 2, the RAG-based summarization and MCQ generation is followed by the email alert, which leads to the answer similarity analysis.

### 3.3.3 OVERALL DIAGRAM

The overall diagram in Figure 3.7 combines the three levels of the data flow diagram into a single flow of the system.



**Figure 3.7. Overall Diagram of the System**

### **3.4 SYSTEM REQUIREMENTS**

#### **3.4.1 HARDWARE REQUIREMENTS**

| <b>Component</b> | <b>Specification</b> |
|------------------|----------------------|
| Processor        | Intel i3 / i5 / i7   |
| RAM              | 8 GB                 |
| Hard Disk        | 500 GB               |

#### **3.4.2 SOFTWARE REQUIREMENTS**

| <b>Component</b>   | <b>Specification</b> |
|--------------------|----------------------|
| Operating System   | Windows 8 / 10       |
| Scripting Language | Python (3.10.8)      |
| Tool               | IDLE (Python)        |

### **3.5 FEASIBILITY STUDY**

The feasibility of the project is analyzed in this phase and a business proposal is put forth with a very general plan for the project and some cost estimates. During system analysis, the feasibility study of the proposed system is carried out to ensure that the proposed system is not a burden to the company. For feasibility analysis, some understanding of the major requirements for the system is essential.

#### **3.5.1 ECONOMICAL FEASIBILITY**

This study is carried out to check the economic impact that the system will have on the organization. The amount of fund that the company can pour into the research and development of the system is limited. The expenditures must be justified. Thus the developed system is well within the budget and this was achieved because most of the technologies used are freely available. Only the customized products had to be purchased.

#### **3.5.2 TECHNICAL FEASIBILITY**

This study is carried out to check the technical feasibility, that is, the technical requirements of the system. Any system developed must not have a high demand on the available technical resources, as this would lead to high demands being placed on

the client. The developed system must have a modest requirement, as only minimal or null changes are required for implementing this system.

### **3.5.3 SOCIAL FEASIBILITY**

This aspect of the study is to check the level of acceptance of the system by the user. This includes the process of training the user to use the system efficiently. The user must not feel threatened by the system, instead must accept it as a necessity. The level of acceptance by the users solely depends on the methods that are employed to educate the user about the system and to make him familiar with it. His level of confidence must be raised so that he is also able to make some constructive criticism, which is welcomed, as he is the final user of the system.

## CHAPTER 4

### PROPOSED SYSTEM ALGORITHM

The proposed system relies on Large Language Models and Retrieval-Augmented Generation for its AI-assisted features and is implemented in the Python programming language. This chapter describes these techniques and technologies.

#### 4.1 LARGE LANGUAGE MODEL (LLM)

Large Language Models (LLMs) are advanced AI systems built on deep neural networks designed to process, understand and generate human-like text.

- LLMs learn patterns, grammar and context from text and can answer questions, write content, translate languages and perform many more tasks.
- By using massive datasets and billions of parameters, LLMs have transformed the way humans interact with technology.
- Modern LLMs include ChatGPT (OpenAI), Google Gemini, Anthropic Claude, etc.

##### 4.1.1 WORKING OF LLM

LLMs are primarily based on the Transformer architecture, which enables them to learn long range dependencies and contextual meaning in text. At a high level, they work through the following steps:

- **Input Embeddings:** Converting text into numerical vectors.
- **Positional Encoding:** Adding sequence/order information.
- **Self-Attention:** Understanding relationships between words in context.
- **Feed-Forward Layers:** Capturing complex patterns.
- **Decoding:** Generating responses step by step.
- **Multi-Head Attention:** Parallel reasoning over multiple relationships.

##### 4.1.2 POPULAR LLMs

- **GPT-4 and GPT-4o (OpenAI):** Advanced multimodal reasoning and dialogue capabilities.
- **Gemini 1.5 (Google DeepMind):** Long-context reasoning, capable of handling 1M+ tokens.

- **Claude 3 (Anthropic):** Safety-focused, strong at reasoning and summarization.
- **LLaMA 3 (Meta):** Open-weight model, popular in research and startups.
- **Mistral 7B / Mixtral (Mistral AI):** Efficient open-source alternatives for developers.
- **BERT and RoBERTa (Google/Facebook):** Strong embedding models for NLP tasks.
- **mBERT and XLM-R:** Early multilingual LLMs.
- **BLOOM:** Large open-source multilingual model, collaboratively developed.

## 4.2 RETRIEVAL-AUGMENTED GENERATION (RAG)

- Retrieval-Augmented Generation (RAG) is an AI technique that combines information retrieval with Large Language Models (LLMs) to generate more accurate and context-relevant responses.
- RAG retrieves relevant information from external documents or knowledge sources before generating an answer.
- RAG helps LLMs use up-to-date and domain-specific information without requiring the model to be retrained.
- RAG is commonly used for document question answering, text summarization, knowledge assistants, and information retrieval applications.

RAG typically consists of two major processes:

1. Retrieval of relevant information
2. Generation of a response based on the retrieved content

### 4.2.1 WORKING OF RAG

RAG works by combining a retrieval system with a generative language model. At a high level, the process works through:

1. Collecting relevant documents, files, or knowledge sources that will be used by the system.

2. Extracting and preprocessing text from uploaded documents and dividing the content into smaller meaningful chunks.
3. Converting document chunks into numerical vector representations that capture their semantic meaning.
4. Storing the generated embeddings in a vector database or vector store for efficient similarity-based retrieval.
5. Converting the user's question or request into an embedding and searching for relevant document chunks.
6. Retrieving the most relevant chunks based on semantic similarity between the query and stored document content.
7. Combining the retrieved information with the user's query and providing it as contextual information to the LLM.
8. The LLM uses the retrieved context to generate a meaningful, relevant, and context-aware response.

#### 4.2.2 POPULAR RAG TECHNOLOGIES AND MODELS

- **LangChain:** Framework commonly used to build RAG-based applications and connect LLMs with documents, retrievers, and vector databases.
- **LlamaIndex:** Framework designed for connecting LLMs with external data sources and building document-based RAG applications.
- **FAISS:** A library developed by Meta for efficient similarity search and retrieval of vector embeddings.
- **ChromaDB:** A vector database commonly used for storing and retrieving document embeddings in AI applications.
- **Pinecone:** A managed vector database designed for high-performance similarity search and retrieval.
- **Weaviate:** An open-source vector database that supports semantic search and AI-powered retrieval.
- **Google Gemini:** A generative AI model that can be integrated into RAG systems to generate responses using retrieved contextual information.
- **GPT Models:** Large Language Models that can use retrieved external context to generate accurate and context-aware responses.

## 4.3 PYTHON PROGRAMMING LANGUAGE

### 4.3.1 INTRODUCTION TO PYTHON

Python is a high-level, object-oriented programming language that was created by Guido van Rossum. It is also called a general-purpose programming language as it is used in almost every domain, such as web development, software development, game development, AI and machine learning, and data analytics.

### 4.3.2 WHY PYTHON?

Every programming language serves some purpose or use-case according to a domain. For example, JavaScript is the most popular language amongst web developers as it gives the developer the power to handle applications via different frameworks like React, Vue and Angular, which are used to build user interfaces. Python, on the other hand, is general-purpose, which means it is widely used in every domain. It is simple to understand and scalable, because of which the speed of development is high, and it does not require any programming background to get started. Python has a simpler syntax similar to the English language, and the syntax allows developers to write programs with fewer lines of code. Since it is open-source, there are many libraries available that make a developer's job easy, ultimately resulting in high productivity and allowing developers to focus on the business logic.

### 4.3.3 REAL-WORLD EXAMPLES

- **NASA:** One of NASA's Shuttle Support Contractors, United Space Alliance, developed a Workflow Automation System (WAS) using Python. NASA has also published a website with about 400 open-source projects which use Python.
- **Netflix:** Several Netflix projects use Python, such as Central Alert Gateway, Chaos Gorilla, Security Monkey and Chronos. The Regional Failover project decreased outage time from 45 minutes to 7 minutes at no additional cost.
- **Instagram:** Instagram uses Python extensively and has built its photo-sharing social platform using Django, a web framework for Python.

#### 4.3.4 APPLICATIONS OF PYTHON

- **Web Development:** Python offers different frameworks such as Django, Pyramid and Flask, which are known for security, flexibility and scalability.
- **Game Development:** PySoy and PyGame are two Python libraries that are used for game development.
- **Artificial Intelligence and Machine Learning:** A large number of open-source libraries are available for developing AI/ML applications.
- **Desktop GUI:** Toolkits and frameworks such as PyQt, PyGtk and PyGUI are available for building desktop applications.

#### 4.3.5 PYTHON STANDARD LIBRARY AND PACKAGES

The Python standard library is very extensive and offers a wide range of facilities. It contains built-in modules, written in C, that provide access to system functionality such as file I/O, as well as modules written in Python that provide standardized solutions for many problems that occur in everyday programming. In addition to the standard library, there is a growing collection of several thousand components, from individual programs and modules to packages and entire application development frameworks, available from the Python Package Index (PyPI).

A script is a file that is executed to accomplish a defined task, a module is a Python program that can be imported into other programs, and a package usually consists of several modules bound together by a package name. Third-party packages are installed from PyPI using the package installer pip, and virtual environments can be used to install the packages required by an application in an isolated location without affecting other applications.

## **CHAPTER 5**

### **IMPLEMENTATION AND RESULT DISCUSSION**

#### **5.1 IMPLEMENTATION**

The implementation of the Student Portal Management System, along with its testing and analysis, is provided in this chapter. The system is implemented as a role-based web application using Python (3.10.8) with IDLE as the development tool on the Windows 8/10 operating system, as listed in Section 3.4.

##### **5.1.1 IMPLEMENTATION OF ROLE-BASED MODULES**

The Admin, Teacher, and Student modules provide separate access to each type of user. The Admin manages student and teacher details, campus interview schedules, and company profiles with eligibility criteria based on 10th and 12th-grade percentages. The Teacher creates and updates timetables and attendance records and prepares question papers with the corresponding answers. The Student views the timetable and attendance details and accesses the AI-based learning and placement services.

##### **5.1.2 IMPLEMENTATION OF DOCUMENT PROCESSING**

The documents uploaded by the students are processed using Retrieval-Augmented Generation as described in Section 4.2. The text of the document is extracted, the relevant content is retrieved, and the LLM generates a concise text summary and multiple-choice questions from the uploaded content.

##### **5.1.3 IMPLEMENTATION OF ANSWER EVALUATION**

Students upload their completed answer-sheet documents, which are processed and compared with the correct answers stored in the database using similarity-based analysis. The system identifies the correctly and incorrectly answered questions and provides an automated evaluation result.

##### **5.1.4 IMPLEMENTATION OF PLACEMENT SERVICES**

The academic percentages of a student are compared with the eligibility requirements stored by the Admin for each company, so that the companies for which the student is eligible can be identified. Email alerts are sent to the students before the scheduled campus interview dates.

Insert the output screenshots of the implemented system here, with captions in the format "Figure 5.1. Caption".

## 5.2 SOFTWARE TESTING

### 5.2.1 UNIT TESTING

Unit testing involves the design of test cases that validate that the internal program logic is functioning properly, and that program inputs produce valid outputs. All decision branches and internal code flow should be validated. It is the testing of individual software units of the application and is done after the completion of an individual unit before integration. This is a structural testing that relies on knowledge of its construction and is invasive. Unit tests perform basic tests at component level and test a specific business process, application, and/or system configuration. Unit tests ensure that each unique path of a business process performs accurately to the documented specifications and contains clearly defined inputs and expected results.

### 5.2.2 INTEGRATION TESTING

Integration tests are designed to test integrated software components to determine if they actually run as one program. Testing is event driven and is more concerned with the basic outcome of screens or fields. Integration tests demonstrate that although the components were individually satisfactory, as shown by successful unit testing, the combination of components is correct and consistent. Integration testing is specifically aimed at exposing the problems that arise from the combination of components.

### 5.2.3 FUNCTIONAL TESTING

Functional tests provide systematic demonstrations that the functions tested are available as specified by the business and technical requirements, system documentation, and user manuals. Functional testing is centered on the following items:

- **Valid Input:** identified classes of valid input must be accepted.
- **Invalid Input:** identified classes of invalid input must be rejected.
- **Functions:** identified functions must be exercised.
- **Output:** identified classes of application outputs must be exercised.
- **Systems/Procedures:** interfacing systems or procedures must be invoked.

Organization and preparation of functional tests is focused on requirements, key functions, or special test cases. In addition, systematic coverage pertaining to identified business process flows, data fields, predefined processes, and successive processes must be considered for testing. Before functional testing is complete, additional tests are identified and the effective value of current tests is determined.

#### **5.2.4 SYSTEM TESTING**

System testing ensures that the entire integrated software system meets requirements. It tests a configuration to ensure known and predictable results. An example of system testing is the configuration-oriented system integration test. System testing is based on process descriptions and flows, emphasizing pre-driven process links and integration points.

#### **5.2.5 WHITE BOX TESTING**

White box testing is a testing in which the software tester has knowledge of the inner workings, structure and language of the software, or at least its purpose. It is used to test areas that cannot be reached from a black box level.

#### **5.2.6 BLACK BOX TESTING**

Black box testing is testing the software without any knowledge of the inner workings, structure or language of the module being tested. Black box tests, as most other kinds of tests, must be written from a definitive source document, such as a specification or requirements document. It is a testing in which the software under test is treated as a black box that cannot be seen into. The test provides inputs and responds to outputs without considering how the software works.

### **5.3 DISCUSSION**

The Student Portal Management System integrates the management of academic, examination, learning, and placement activities in a single platform. The implementation of the role-based modules reduces the manual maintenance of student records, timetables, attendance, and company details. The RAG-based document processing allows students to obtain summaries and multiple-choice questions from their own study materials, and the resume-based eligibility analysis together with the email alerts makes the placement activities more organized.

The similarity-based answer evaluation compares the student answers with the stored answers and identifies the correctly and incorrectly answered questions. The effectiveness of this analysis is expected to be approximately 80–90%, depending on the document quality and the answer similarity. Overall, the system reduces manual work, improves academic monitoring, and supports AI-assisted learning.

## **CHAPTER 6**

### **CONCLUSION AND FUTURE ENHANCEMENT**

#### **6.1 CONCLUSION**

The Student Portal Management System provides a centralized platform for managing academic, examination, student, teacher, and campus placement activities efficiently. The system simplifies timetable and attendance management, question paper preparation, student document processing, and answer-sheet evaluation. The integration of RAG enables students to generate meaningful summaries and MCQs from uploaded study materials, supporting AI-assisted learning. Resume-based eligibility analysis helps students identify suitable placement opportunities, while email notifications keep them informed about upcoming campus interviews. Overall, the system reduces manual work, improves academic monitoring, and provides an integrated and intelligent solution for student management and placement coordination.

#### **6.2 FUTURE ENHANCEMENT**

In the future, the system can be enhanced by integrating advanced AI models to provide more accurate document summarization, MCQ generation, and answer evaluation. A chatbot can be added to help students with academic and placement-related queries. The placement module can be extended with job recommendation features based on students' skills, academic performance, and resumes. Mobile application support can also be introduced to provide students and teachers with easy access to the system from anywhere. In addition, advanced analytics and dashboards can be implemented to track student performance, attendance, examination results, and placement progress.

## REFERENCES

- [1] T. Nguyen, Q. Pham, L. Chen, and R. Zhang, “A review on personalized learning and intelligent tutoring systems using deep learning and graph-based approaches,” *IEEE Access*, 2023. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/11386890/>
- [2] A. A. Syed, F. L. Gaol, and T. Matsuo, “A survey of the state-of-the-art models in neural abstractive text summarization,” *IEEE Access*, vol. 9, pp. 13248–13265, 2021, doi: 10.1109/ACCESS.2021.3052783.
- [3] A. Elsaid, A. Mohammed, L. F. Ibrahim, and M. M. Sakre, “A comprehensive review of Arabic text summarization,” *IEEE Access*, vol. 10, pp. 38012–38030, 2022, doi: 10.1109/ACCESS.2022.3163292.
- [4] S. Tanwar, Q. Bhatia, P. Patel, A. Kumari, P. K. Singh, and W.-C. Hong, “Machine learning adoption in blockchain-based smart applications: The challenges, and a way forward,” *IEEE Access*, vol. 8, pp. 474–488, 2020, doi: 10.1109/ACCESS.2019.2961372.
- [5] B. Khan, R. Naseem, F. Muhammad, G. Abbas, and S. Kim, “An empirical evaluation of machine learning techniques for chronic kidney disease prophecy,” *IEEE Access*, vol. 8, pp. 55012–55022, 2020, doi: 10.1109/ACCESS.2020.2981689.
- [6] C. W. Tan, L. Ling, P.-D. Yu, C. N. Hang, and M. F. Wong, “Mathematics gamification in mobile app software for personalized learning at scale,” in *Proc. IEEE Integr. STEM Educ. Conf. (ISEC)*, Aug. 2020, pp. 1–5.
- [7] D. R. Ch and S. K. Saha, “Automatic multiple choice question generation from text: A survey,” *IEEE Trans. Learn. Technol.*, vol. 13, no. 1, pp. 14–25, Jan. 2020.
- [8] Y. Chen, W. Huang, and K.-S. Lee, “Modeling knowledge acquisition with graph neural networks in educational systems,” *IEEE Trans. Learn. Technol.*, 2024. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/11386890/>
- [9] F. K. Sufi and I. Khalil, “Automated disaster monitoring from social media posts using AI-based location intelligence and sentiment analysis,” *IEEE Trans.*

*Computat. Social Syst.*, early access, Mar. 18, 2022, doi: 10.1109/TCSS.2022.3157142.

- [10] R. Pramana, Debora, J. J. Subroto, A. A. S. Gunawan, and Anderies, “Systematic literature review of stemming and lemmatization performance for sentence similarity,” in *Proc. IEEE 7th Int. Conf. Inf. Technol. Digit. Appl. (ICITDA)*, Nov. 2022, pp. 1–6.